
Práctica 5: Listas Doblemente Ligadas

Estructura de Datos 2027-1

Elaboró

Erik Quintero Villeda

Objetivos

Implementar el tipo de dato *Lista Doblemente Ligada* por medio de *TDA Lista* y *TDA Colección*; y utilizar dicha implementación para resolver un problema.

Instrucciones:

1. Implementa los métodos faltantes en la clase `ListaDoblementeLigada` tomando en cuenta la **nota 10** del curso:

- a) `agregar (elemento: T):`
Agrega *elemento* al inicio de una lista.
- b) `eliminar (elemento: T):`
Elimina la primera aparición de *elemento* en una lista.
- c) `buscar (elemento: T):`
Determina si *elemento* se encuentra en una lista.
- d) `eliminar_indice (i: int):`
Elimina el elemento ubicado en la *i*-ésima posición de una lista.
- e) `acceder (i: int):`
Devuelve el elemento ubicado en la *i*-ésima posición de una lista.
- f) `devolver_indice_elemento (elemento: T):`
Devuelve el índice de la primera aparición de *elemento* en una lista.
- g) `devolver_longitud():`
Devuelve la longitud de una lista.
- h) `agregar_final (elemento: T):`
Agrega *elemento* al final de una lista.
- i) `reversa():`
Devuelve una nueva lista con los elementos en orden inverso respecto a la lista original.
- j) `acceder_nodo (i: int):`
Devuelve el *i*-ésimo nodo de una lista.
- k) `__str__ ():`
Devuelve la representación en cadena de una lista doblemente ligada con el siguiente formato:

$$a_1 \leftrightarrow a_2 \leftrightarrow \dots \leftrightarrow a_n$$

La clase `MainListaDoble` contiene un método `main` que se utilizará para probar cada uno de los métodos implementados. Los resultados se imprimirán en la terminal.

2. Utilizando tu implementación de listas doblemente ligadas, resuelve el siguiente problema:

Problema:

Te contratan como programador para desarrollar un nuevo navegador llamado *El Chido*. Tu trabajo consiste en implementar una versión muy básica del historial de navegación. Para ello, recibes un archivo de texto (`txt`) que contiene los nombres de todas las páginas que visitó un usuario junto con la fecha de acceso.

- Los registros de las páginas están ordenados cronológicamente; es decir, las primeras líneas corresponden a las páginas visitadas primero.
- Cada línea del archivo tiene el siguiente formato:

`<nombre_página>: <fecha_acceso>`

El historial debe cumplir con los siguientes requisitos:

- a) La primera página almacenada debe ser la más reciente, y así sucesivamente hasta la más antigua.
- b) El historial no debe contener páginas repetidas. En caso de que una página aparezca más de una vez en el archivo, solo se debe almacenar la versión con la fecha más reciente.

Los archivos correspondientes a esto último estarán en el repositorio.

Deberás programar una clase con un `Main` la cual imprima en pantalla el historial correspondiente a un `txt`. Además, deberás programar una clase `Historial` y una clase `Pagina` que representen las entidades involucradas en el problema.

Reporte

1. Elabora un reporte donde realices lo siguiente:
 - a) Describe las principales complicaciones que tuviste al desarrollar la práctica.
 - b) Explica las diferencias entre los métodos `agregar`, `eliminar`, `buscar` y `acceder` con respecto a los métodos equivalentes programados en la Práctica 4.
 - c) Explica la utilidad de programar las clases `Pagina` e `Historial` y las desventajas en caso de haberlas. Si consideras que no existe ninguna de estas, explica por qué.
 - d) Explica cuál es el proceso que sigue el algoritmo implementado en el método `agregar_pagina` que se encuentra en la clase `Historial`.
2. Además, responde las siguientes preguntas:
 - a) ¿Cuáles son las principales ventajas de esta implementación de listas doblemente ligadas en comparación con la implementación de listas ligadas simples de la Práctica 4? Explica de manera general.

- b) ¿Por qué el atributo principal de la clase `Historial` es de tipo `Lista` en lugar de `ListaDoblementeLigada`?
 - c) ¿Consideras que es más útil resolver el problema del historial usando listas ligadas simples?
 - d) ¿Cuáles son los cambios mínimos que deberías hacer en tu programa para resolver el problema usando listas ligadas simples?
3. Pídele a tu IA de confianza que responda las preguntas anteriores y anexa las respuestas al reporte. Agrega un breve **análisis crítico** donde expongas las principales diferencias entre las respuestas de la IA y las tuyas. De ser posible señala los puntos clave.

Requisitos de entrega:

Los requisitos que deben cumplir las entregas son los siguientes:

1. La entrega se realizará únicamente a través del Classroom. El código se debe encontrar comprimido en un archivo `.zip` con el nombre `[EDD-7063] P5 <apellido(s)_nombre(s)>`.
2. Deberán incluir en el archivo `README.md` las instrucciones para compilar y ejecutar el código.
3. La práctica podrá entregarse en parejas. Deberán escribir el nombre de los integrantes en el archivo `README.md` y en el reporte, así como su número de cuenta.
4. No se aceptarán prácticas que no compilen.
5. Se permite añadir métodos privados auxiliares para resolver la práctica. Sin embargo, no está permitido eliminar o agregar atributos ni constructores en las clases proporcionadas.
6. No está permitido modificar la firma de los métodos proporcionados en el esqueleto del código.
7. No podrán modificarse los métodos que ya estén implementados en las clases proporcionadas.
8. El reporte deberá entregarse en formato `.pdf` y no debe tener faltas de ortografía. El reporte debe ir fuera del archivo `.zip` nombrado `[EDD-7063] Reporte P5 <apellido(s)_nombre(s)>`.
9. Por cada día de retraso en la entrega de la práctica se restarán 2 puntos a la calificación obtenida. Las prácticas con más de dos días de retraso no se aceptarán.
10. No está permitido utilizar ninguna biblioteca externa (si lo consideras necesario, deberás consultarlo al equipo escribiendo un comentario privado en Classroom).

Evaluación

Práctica implementada de forma completa y en correcto funcionamiento	60 %
Entrega del reporte contestado correctamente en el formato establecido	30 %
Documentación del código (<code>DocString</code> , <code>pydoc</code>)	10 %